DIPLOMA SEMESTER 2 • PRACTICAL WORKSHOP • 45 PERIODS

Electronics Tinkering Workshop

This is a practical handbook for Course 2049. It is built like a workshop manual: what to identify, what to do, how to do it safely, what observation to record, what output is acceptable, and how to prepare for lab examination and viva.

COURSE CODE
2049

CREDITS
No Credit

PATTERN
L:0 T:0 P:3

TOTAL PERIODS
45

OFFICIAL SYLLABUS STRUCTURE

Course overview and outcome map

The course starts from soldering practice, then moves to rework and cable termination, and finally uses Arduino and Raspberry Pi for live interfacing activities. It does not expect prior C or Python mastery; the practical focus is modification, connection, testing, and documentation.

Course objectives converted into workshop actions

- ✓ Develop safe hands-on skill for simple hobby projects.
- ✓ Use electronic components, Arduino, Raspberry Pi and compatibles.
- ✓ Learn through real-time practical experience rather than theory-only programming.
- ✓ Follow coding standards: comments, proper naming, and acknowledgement when example code is modified.

> □ □□□□□ □□□ theory note □□□□. □□ □□□□□□ solder □□□□□□□, circuit build □□□□□□□, Arduino/Raspberry Pi connect □□□□□□□, output verify □□□□□□□ □□□□□□□□ □□□□□□ □□□□□□□.

Course outcomes and periods

CO	Outcome	Periods	Level
CO1	Construct simple models and hobby circuits by soldering techniques.	9	Applying
CO2	Apply soldering skill to dismantle/rework simple electronic circuits.	9	Applying
CO3	Develop simple interfacing applications using Arduino.	15	Applying
CO4	Develop simple interfacing applications using Raspberry Pi.	9	Applying
Exam	Lab examinations and assessment.	3	Practical

How to use this handbook in lab

1. Read the safety section before touching soldering iron, PCB, Arduino, Raspberry Pi or supply.
2. For every practical, write Aim, Components/Tools, Procedure, Connection/Circuit, Observation, Result and Faults corrected.

3. Do not copy example code blindly. First run the basic example, then change delay, pin number, message, sensor threshold or output action.
4. In every Arduino/Raspberry Pi task, prove the output by observation: LED blink, relay click, LCD text, motor movement, GPIO voltage or terminal print.
5. Use the viva and lab exam sections as final checklist before assessment.

BENCH PREPARATION

Tools, equipment and consumables

Good workshop output depends more on tool control than expensive components. A weak solder joint, wrong crimp or loose jumper wire can waste more time than a wrong program.

Soldering and rework tools

Soldering iron of suitable wattage, temperature-controlled station, soldering iron stand, fume extractor, solder wire, flux, tip cleaner, nipper, wire stripper, needle-nose plier, tweezer, desoldering pump and rework station.

PCB and assembly material

Through-hole PCB, dot board, prefabricated hobby kit PCB, single-strand copper wire, hook-up wire, heat-shrink sleeve, resistors, LEDs, capacitors, headers, terminal blocks and small power supply modules.

Arduino section

Arduino UNO or compatible, USB cable, Arduino IDE, relay module, PIR sensor, LCD shield, Bluetooth HC-05/HC-06, motor driver shield, robot chassis, DC geared motors, caster wheel, 9 V battery pack and line sensor.

Raspberry Pi section

Raspberry Pi or compatible board, microSD card, power adapter, OS image, display/SSH access, GPIO jumpers, breadboard, LEDs, resistors, push buttons and simple input-output loads.

Brutal truth: If the bench is messy, the practical result will be unreliable. Most student circuit failures are not because of advanced electronics; they are because of wrong polarity, dry solder, shorted tracks, loose jumper wires, missing ground, or wrong supply voltage.

MANDATORY

Safety, ESD and quality rules

This workshop uses heat, fumes, sharp cutters, mains-powered adaptors and sensitive semiconductor boards. Safety must be treated as part of the output, not as a separate formality.

Personal safety

- ✓ Keep soldering iron in the stand when not in hand.
- ✓ Do not touch iron tip, molten solder or recently soldered leads.
- ✓ Use fume extraction or work in a ventilated area.
- ✓ Cut component leads away from the face and other students.
- ✓ Keep water, bags and loose paper away from the bench.

Component and PCB safety

- ✓ Confirm resistor value, capacitor polarity, diode direction and IC orientation before soldering.
- ✓ Use ESD precautions for microcontroller boards and ICs.
- ✓ Do not overheat pads. Heat the pad and lead together, then feed solder.
- ✓ Never power Arduino or Raspberry Pi from unknown voltage.
- ✓ Always connect common ground when interfacing modules.

Stop-and-check conditions

Stop the practical and call faculty/instructor if you see smoke, smell burning insulation, feel overheating on a regulator, observe repeated USB disconnection, find a short across supply rails, or hear abnormal motor/relay chattering.

CO1 • 9 PERIODS

Construct simple models and hobby circuits by soldering techniques

Goal: identify tools, understand safe use, perform tinning, make copper mesh models, assemble kit PCBs and build a simple dot-board circuit.

Soldering fundamentals

A good solder joint is shiny, smooth, mechanically stable, electrically continuous and does not bridge nearby pads. Solder is not glue; the joint must be supported by correct component lead placement.

> Good joint = clean pad + clean lead + correct heat + correct solder + no movement while cooling

Correct soldering sequence

1. Clean the copper pad and component lead.
2. Tin the iron tip lightly.
3. Place the component and bend leads slightly.
4. Touch tip to both pad and lead.
5. Feed solder into the heated joint, not directly onto the tip only.
6. Remove solder wire first, then remove iron.
7. Do not move the joint until solidified.
8. Trim excess lead after inspection.

Common soldering defects

Defect	Appearance	Cause	Correction
Dry joint	Dull, grainy, weak	Low heat or movement	Reheat with flux and fresh solder

Defect	Appearance	Cause	Correction
Solder bridge	Short between pads	Excess solder	Remove using wick/pump
Lifted pad	Pad detached	Overheating or force	Use jumper repair if allowed
Cold lead	Solder sits around lead	Lead not heated	Heat lead and pad together
Flux residue	Sticky brown surface	Uncleaned flux	Clean with approved cleaner

Practical: 2 × 2 copper wire mesh

1. Cut equal lengths of single-strand copper wire.
2. Straighten and tin all ends.
3. Place two horizontal and two vertical wires as a square mesh.
4. Solder each crossing after confirming alignment.
5. Inspect mechanical strength by gentle touch only.

> Mesh work soldering control □□□□□□□□ □□□. Alignment □□□□□□□□□ circuit soldering-□□□ same mistake repeat □□□□□□□.

Practical: 2 × 2 × 2 cube

1. Prepare 12 equal wire pieces for cube edges.
2. Tin all joining ends before assembly.
3. Make two square frames first.
4. Join the frames using four vertical members.
5. Check that the cube stands without twist.

Record observation: mention wire length, number of joints, soldering defects corrected, and final mechanical stability.

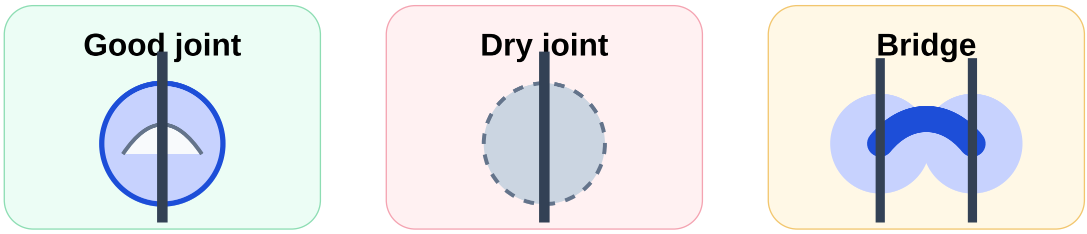
Hobby PCB and dot-board circuit guide

Kit PCB method

1. Verify PCB print and component list.
2. Start with low-height components: resistors, diodes, jumpers.
3. Place IC socket before IC. Never overheat IC pins.
4. Place electrolytic capacitors with correct polarity.
5. Mount connectors and switches last.
6. Inspect solder side before power ON.

Dot-board method

1. Draw the layout before soldering.
2. Keep power rails clear and labelled.
3. Use short jumpers and avoid crossing where possible.
4. Test continuity after each major connection.
5. Mark final input, output, VCC and GND.



Visual inspection targets for soldering practicals.

CO2 • 9 PERIODS

Dismantling, rework, wire joining, crimping and cable tagging

Goal: use soldering skill for controlled removal and repair, then apply practical wiring methods used in panels, instruments and prototype circuits.

Through-hole desoldering method

1. Identify the component and mark polarity/orientation before removing.
2. Add small amount of fresh solder if the old joint is oxidised.
3. Heat the joint fully and use desoldering pump/wick.
4. Do not pull component before solder is molten.
5. Clean hole, inspect pad, then remove component.
6. Check continuity of remaining track after rework.

Wire joining quality

Wire joints must be electrically continuous, mechanically strong and insulated. A joint hidden under heat-shrink sleeve is still wrong if twisting/tinning was poor.

Joint	Use	Important point
Straight joint	Extending same line	Overlap, twist, solder and sleeve
Branch joint	Tap connection	Main conductor should not be weakened
Crimp terminal	Panel/terminal block wiring	Correct ferrule/lug and crimp tool
Cable tag	Identification	Readable, consistent, near termination

Crimping checklist

- ✓ Strip correct length only.
- ✓ No copper strands cut.

- ✓ Ferrule/lug size matches conductor.
- ✓ Insulation not inside conductor crimp zone.
- ✓ Pull test passed.

Cable tagging checklist

- ✓ Tag number matches circuit drawing.
- ✓ Text direction is readable.
- ✓ Tag does not cover stripped copper.
- ✓ Same naming convention used throughout.
- ✓ Tag survives gentle handling.

Rework mistakes

- ✓ Overheating pad until it lifts.
- ✓ Pulling component lead while solder is solid.
- ✓ Leaving solder balls on PCB.
- ✓ Wrong polarity after replacement.
- ✓ Skipping continuity test.

Panel wiring relevance

For an electrical/electronics drawing engineer, this module is not just a college exercise. Crimping, cable numbering, wire joining, sleeve selection and termination inspection are directly connected to controller-panel wiring quality, troubleshooting speed and service reliability.

CO3 • 15 PERIODS

Arduino interfacing applications

Goal: use Arduino UNO and compatibles to develop simple interfacing tasks by using example programs, modifying them and verifying the real output.

Arduino UNO minimum knowledge

- ✓ Digital pins can be used as INPUT or OUTPUT.
- ✓ P13 has onboard LED on many UNO boards.
- ✓ 5 V and GND must be correctly connected.
- ✓ USB is used for programming and serial monitor.
- ✓ Do not drive motor or relay coil directly from an Arduino I/O pin.

> Arduino practical-□ code copy □□□□□□□□□□□□ □□□□□□□ □□□□□□□□. Pin number, input/output direction, power supply, common GND □□□□□□□ verify □□□□□□□□.

Basic Arduino program structure

```
void setup() {
  pinMode(13, OUTPUT);   // runs once
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Modify delay values, move LED to another pin using a resistor, and record the output difference.

Practical 1: LED blink and relay switching

1. Connect LED/relay module to the selected digital pin.
2. Set pinMode as OUTPUT.
3. Switch output HIGH and LOW at different intervals.
4. Observe LED blinking or relay clicking.
5. Change ON time and OFF time and note effect.

Relay warning: Use a relay module with driver transistor/opto isolation. Never connect relay coil directly to an Arduino I/O pin.

Practical 2: PIR motion sensor with relay

1. Connect PIR VCC, GND and OUT correctly.
2. Read PIR OUT using digital input.
3. If motion is detected, switch relay output ON.
4. If no motion is detected, switch relay output OFF after required delay.
5. Record sensor range, delay and false trigger observations.

Practical 3: DC motor shield

Use a DC motor control shield to run motor forward, reverse and stop. The important point is not only rotation; direction command and supply rating must match the motor driver shield.

- ✓ Check motor supply voltage.
- ✓ Check common ground.
- ✓ Do not stall motor for long time.
- ✓ Record forward/reverse truth table.

Practical 4: Line follower robot

A line follower robot uses two geared DC motors, a caster wheel, Arduino UNO, motor driver shield, battery and line sensor. The control logic is simple: detect left/right line condition and adjust motor direction/speed.

> If left sensor detects line → correct toward left. If right sensor detects line → correct toward right. If both aligned → move forward.

Practical 5: Bluetooth control

Use HC-05/HC-06 Bluetooth module to receive commands from a smartphone and switch relay or robot movement. Maintain proper voltage level and pairing settings.

```
// Typical command idea
// F = forward, B = backward, L = left, R = right, S = stop
if (cmd == 'F') forward();
else if (cmd == 'S') stopRobot();
```

Practical 6: LCD shield

Display your name, class number and experiment title. In the record, include LCD type, library used, pins/shield used, and final displayed text.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(8, 9, 4, 5, 6, 7);
void setup(){
  lcd.begin(16, 2);
  lcd.print("Name: Nanda");
  lcd.setCursor(0,1);
  lcd.print("Class No: 01");
}
void loop(){}
```

Arduino practical record template

Heading	What to write
Aim	State the exact output: blink LED, switch relay, display text, control motor, detect motion.
Components	Arduino board, module name, resistor value, supply, wires, shield, sensor.
Connection	Pin mapping table: Arduino pin → component pin → purpose.
Program	Paste only final tested program with comments.
Observation	What changed physically after upload: LED status, relay click, motor direction, LCD text.
Faults corrected	Wrong pin, loose wire, missing GND, wrong COM port, wrong library etc.

CO4 • 9 PERIODS

Raspberry Pi interfacing applications

Goal: become familiar with Raspberry Pi board, install OS, use basic operating system features and interface GPIO for input/output applications.

Raspberry Pi essentials

- ✓ Raspberry Pi is a single-board computer, not just a microcontroller board.
- ✓ It needs an operating system on microSD card.
- ✓ GPIO pins are not 5 V tolerant on many models; use proper voltage levels.
- ✓ Linux terminal use is part of the practical skill.
- ✓ Shutdown properly before removing power.

OS installation checklist

1. Download/use suitable Raspberry Pi OS image.
2. Write image to microSD card using approved tool.
3. Insert microSD card and connect display/keyboard or SSH setup.
4. Power using recommended adaptor.
5. Complete basic configuration.
6. Test terminal and GPIO library availability.

GPIO LED output idea

```
from gpiozero import LED
from time import sleep
led = LED(17)
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

Record BCM pin number, physical pin number, resistor value and observed blink rate.

GPIO input-output idea

```
from gpiozero import Button, LED
from signal import pause
button = Button(2)
led = LED(17)
button.when_pressed = led.on
button.when_released = led.off
pause()
```

Use pull-up/pull-down correctly. Do not leave input floating.

Raspberry Pi DIY project examples

GPIO status panel

Read switches and display LED status. Useful for understanding input/output logic similar to control-panel indication.

Mini temperature logger

Use a compatible sensor module and log readings with timestamp. Focus on wiring, library use and data recording.

Relay automation demo

Switch a low-voltage load through relay module. Include isolation and supply notes.

Network display

Show board IP address or status message. Useful for learning Raspberry Pi as a small controller/computer.

DOCUMENTATION

Practical record format and observation discipline

Marks are lost when students build correctly but record poorly. Record must prove that you understood the connection, safety, output and correction.

Common record format

1. Experiment number and date.
2. Aim.
3. Tools/components required.
4. Circuit/connection diagram or pin table.
5. Procedure.
6. Program, if applicable.
7. Observation table.
8. Result and troubleshooting notes.
9. Signature/verification.

Observation table examples

Experiment	Observation to record
Solder mesh	Number of joints, defects corrected, mechanical stability.
LED kit PCB	Supply voltage, LED status, polarity corrections.
Wire crimping	Wire size, terminal type, pull-test result.
Arduino PIR	Motion/no motion output and relay state.
Raspberry Pi GPIO	Pin number, voltage level, LED/button response.

Bad record practices

Do not write “output verified” without saying what output. Do not paste random internet code without comments. Do not draw a connection diagram that does not match the actual pins used. Do not skip troubleshooting; corrected mistakes show real learning.

DIY

Open-ended project ideas

Choose projects that can be built, tested and explained within workshop time. Fancy project names are useless if you cannot explain input, processing, output and power supply.

Arduino night lamp

LDR input + relay/LED output. Learn threshold decision and output switching.

PIR visitor alert

PIR sensor + buzzer/relay. Learn digital input and timed output.

Bluetooth controlled lamp demo

HC-05/HC-06 + relay module. Learn smartphone command control.

Mini line follower

Robot kit + line sensor + motor shield. Learn closed-loop correction logic.

Raspberry Pi GPIO dashboard

Show switch input state and LED output state from Python program.

Workshop tool counter

Simple button input count display/log. Useful for tool issue-return monitoring concept.

VIVA BANK

Important viva and short-answer questions

Prepare these with practical examples. One-line textbook answers are not enough for a workshop subject.

1. What is tinning and why is it done before soldering?
2. List five tools used for through-hole soldering and desoldering.
3. Why is a fume extractor recommended in soldering practice?
4. What is ESD safety? Name two ESD precautions.
5. Differentiate dry joint and solder bridge.
6. Why should an IC socket be used in a hobby PCB?
7. What is the use of heat-shrink sleeve in wire joining?
8. Why should stranded wire not be cut while stripping insulation?
9. What is the purpose of cable tags?
10. State two checks after crimping a cable terminal.
11. What is Arduino UNO?
12. What is the use of pinMode() in Arduino?
13. Why is common ground required between Arduino and external modules?
14. Why should a relay coil not be connected directly to an Arduino pin?
15. What is PIR sensor used for?
16. What is HC-05/HC-06 module?
17. What is the function of motor driver shield?
18. Name the main parts of a line follower robot.
19. What is the use of LCD shield?

20. How is Raspberry Pi different from Arduino?

21. Why does Raspberry Pi need an operating system?

22. What is GPIO?

23. Why should Raspberry Pi be shut down properly?

24. What details must be written in a practical record?

25. What is the correct troubleshooting order when a circuit does not work?

Suggested short answers

Tinning means coating the wire/lead/tip with thin solder before final joining. It improves heat transfer and joint quality. ESD means electrostatic discharge; wrist strap, antistatic mat and safe handling protect sensitive devices. Arduino is a microcontroller development board; Raspberry Pi is a single-board computer running an OS. GPIO means General Purpose Input/Output. Common ground gives the same voltage reference to modules and controller.

ASSESSMENT

Lab examination pattern and rubrics

For this practical workshop, the useful exam model is a task sheet plus viva and record verification. The focus must be actual bench skill.

Possible lab exam task sheet

Part A: Soldering / Rework Task
1. Identify soldering and desoldering tools.
2. Perform tinning of given wire.
3. Make a proper solder joint or remove a component from a through-hole PCB without damaging pad.
4. Inspect and correct one defective joint.

Part B: Wiring Task
5. Prepare one straight wire joint with heat-shrink sleeve OR crimp a terminal using proper tool.
6. Apply correct cable tag and perform pull test.

Part C: Arduino / Raspberry Pi Task
7. Upload or modify an example program to blink LED / switch relay / read sensor / display text.
8. Show output to examiner and explain pin connections.

Part D: Viva and Record
9. Answer safety, tool, circuit and coding questions.
10. Submit completed practical record.

Marking rubric

Area	Marks focus
Safety	Correct handling of iron, tools, ESD and power.
Build quality	Solder joint, wiring, crimp, insulation and neatness.
Function	Required output works repeatedly.
Troubleshooting	Finds wiring/code/power mistake logically.
Record	Connection table, observation and result are complete.
Viva	Explains what was done, not memorised blindly.

Before submitting a practical

- ✓ Bench is clean and safe.
- ✓ Power is OFF before circuit changes.
- ✓ PCB/wiring is inspected under good light.
- ✓ Pin mapping matches actual connection.
- ✓ Program is final tested version with comments.
- ✓ Observation table contains real measured/seen output.

QUICK REVISION

One-page memory sheet

CO1

Soldering iron, stand, fume extractor, solder, flux, nipper, stripper, tweezer, desoldering pump, PCB, SMD/through-hole, tinning, copper mesh, kit PCB, dot board.

CO2

Desolder carefully, avoid lifted pad, sleeve wire joints, crimp with correct terminal, tag cables clearly, test continuity and pull strength.

CO3

Arduino UNO, IDE, P13 LED, relay, PIR, motor shield, line follower robot, Bluetooth HC-05/06, LCD shield, DIY project.

CO4

Raspberry Pi, OS installation, Linux basics, GPIO input/output, Python examples, safe voltage, proper shutdown, DIY project.

Core troubleshooting order

1. Check power supply and ground.
2. Check component polarity and pin mapping.
3. Check physical joint or jumper continuity.
4. Check code pin numbers and libraries.
5. Test one block at a time.
6. Record the fault and correction.

REFERENCES

Official online resources from syllabus

No.	Resource	Use in workshop
1		IDE, board documentation, examples and libraries.
2		Component tutorials, modules and practical guides.

No.	Resource	Use in workshop
4		